🚀 Minishell Development Plan - Two Person Team

👥 Team Roles Division

Person A: Parser/Tokenizer Specialist

- **Focus**: Input processing, syntax analysis, command structure building
- **Core Responsibility**: Transform user input into executable command structures

Person B: Execution/Functions Specialist (You)

- **Focus**: Command execution, pipes, redirections, builtins, system calls
- **Core Responsibility**: Execute parsed commands with proper process management

---

📋 Phase 1: Foundation & Setup (Week 1)

🤝 Together - Project Setup

- [ ] **Create project structure**

```
minishell/
├── src/
│   ├── parsing/
│   ├── execution/
│   ├── builtins/
│   └── utils/
├── includes/
├── libft/
└── Makefile
```

- [ ] **Define shared header** `minishell.h`
- [ ] **Agree on core data structures**
- [ ] **Setup git repository with proper branching strategy**
- [ ] **Create basic Makefile**

🔍 Person A Tasks - Parsing Foundation

- [ ] **Design token types enum**

```c
typedef enum e_token_type {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_REDIRECT_IN,
    TOKEN_REDIRECT_OUT,
    TOKEN_REDIRECT_APPEND,
    TOKEN_HEREDOC,
    TOKEN_EOF
} t_token_type;
```

- [ ] **Create tokenizer structure**
- [ ] **Implement basic lexer (split input into tokens)**
- [ ] **Handle quotes and escaping**

⚙️ Person B Tasks - Execution Foundation

- [ ] **Refine pipeline execution functions** (based on your current code)
- [ ] **Implement PATH resolution** (using the refactored code)
- [ ] **Create basic builtin framework**
- [ ] **Test pipe creation and process management**

🎯 **Checkpoint 1**: Can tokenize simple commands and execute basic programs

---

📋 Phase 2: Core Parsing & Basic Execution (Week 2)

🔍 Person A Tasks - Advanced Parsing

- [ ] **Implement parser (tokens → AST/command list)**
- [ ] **Handle pipe parsing** (`cmd1 | cmd2 | cmd3`)
- [ ] **Handle basic redirections** (`cmd > file`, `cmd < file`)
- [ ] **Create command structure builder**
- [ ] **Add syntax error detection**

⚙️ Person B Tasks - Core Builtins

- [ ] **Implement `echo` with `-n` flag**
- [ ] **Implement `pwd`**
- [ ] **Implement `env`**
- [ ] **Implement basic `cd` (home directory)**
- [ ] **Create builtin dispatcher**

🤝 Together - First Integration

- [ ] **Define interface between parsing and execution**
- [ ] **Test simple command execution**: `ls`, `pwd`, `echo hello`
- [ ] **Test basic pipes**: `ls | grep txt`

🎯 **Checkpoint 2**: Can parse and execute simple commands and pipes

---

📋 Phase 3: Advanced Features (Week 3)

🔍 Person A Tasks - Complex Parsing

- [ ] **Implement heredoc parsing** (`cmd << delimiter`)
- [ ] **Handle append redirection** (`cmd >> file`)
- [ ] **Improve quote handling** (nested quotes, escaping)
- [ ] **Add variable expansion** (`$USER`, `$HOME`)
- [ ] **Handle multiple redirections per command**

⚙️ Person B Tasks - Advanced Execution

- [ ] **Implement heredoc execution** (using your improved version)
- [ ] **Handle file redirections** (create, truncate, append)
- [ ] **Implement `export` builtin**
- [ ] **Implement `unset` builtin**
- [ ] **Add proper error handling and exit codes**

🤝 Together - Advanced Integration

- [ ] **Test complex redirections**: `cmd < input > output`
- [ ] **Test heredoc**: `cat << EOF`
- [ ] **Test environment manipulation**

🎯 **Checkpoint 3**: Handles most shell features except signals

---

📋 Phase 4: Polish & Edge Cases (Week 4)

🔍 Person A Tasks - Edge Cases

- [ ] **Handle empty lines and whitespace**
- [ ] **Improve error messages for syntax errors**
- [ ] **Handle special characters properly**
- [ ] **Add command line history (bonus)**

⚙️ Person B Tasks - System Polish

- [ ] **Implement `exit` builtin with status codes**
- [ ] **Handle signals (`Ctrl+C`, `Ctrl+\`, `Ctrl+D`)**
- [ ] **Fix memory leaks and fd leaks**
- [ ] **Add comprehensive error handling**

🤝 Together - Final Integration

- [ ] **Extensive testing with complex command combinations**
- [ ] **Memory leak testing with valgrind**
- [ ] **Performance optimization**
- [ ] **Code review and refactoring**

🎯 **Checkpoint 4**: Production-ready minishell

📁 File Organization Strategy

🏗️ Core Structure

```
src/
├── parsing/
│   ├── tokenizer.c
│   ├── parser.c
│   ├── quotes.c
│   └── expansion.c
├── execution/
│   ├── pipeline.c
│   ├── redirections.c
│   ├── path_resolution.c
│   └── process_management.c
├── builtins/
│   ├── builtin_manager.c
│   ├── echo.c
│   ├── cd.c
│   ├── pwd.c
│   ├── env.c
│   ├── export.c
│   ├── unset.c
│   └── exit.c
├── utils/
│   ├── error_handling.c
│   ├── memory_management.c
│   └── string_utils.c
└── main.c
```

📝 Shared Data Structures

```c
// In minishell.h
typedef struct s_token {
    char *value;
    t_token_type type;
    struct s_token *next;
} t_token;

typedef struct s_cmd {
    char **args;        // Command and arguments
    char *input_file;   // Input redirection file
    char *output_file;  // Output redirection file
    int input_fd;       // Input file descriptor
    int output_fd;      // Output file descriptor
    int append_mode;    // Append flag for output
    char *heredoc_delimiter; // For heredoc
    struct s_cmd *next; // Next command in pipeline
} t_cmd;

typedef struct s_shell {
    char **envp;        // Environment variables
    int exit_status;    // Last command exit status
    int should_exit;    // Flag to exit shell
} t_shell;
```

🔧 Integration Strategy

🤝 Interface Design

```c
// Person A provides:
t_cmd *parse_input(char *input, t_shell *shell);
void free_cmd_list(t_cmd *cmd_list);

// Person B provides:
int execute_pipeline(t_cmd *cmd_list, t_shell *shell);
int is_builtin(char *cmd);
int execute_builtin(t_cmd *cmd, t_shell *shell);
```

🔄 Main Integration Loop

```c
int main(int argc, char **argv, char **envp)
{
  t_shell shell;
  char *input;
  t_cmd *cmd_list;

  init_shell(&shell, envp);

  while (!shell.should_exit)
  {
    input = readline("minishell$ ");
    if (!input)
      break;

    if (*input)
      add_history(input);

    cmd_list = parse_input(input, &shell); // Person A
    if (cmd_list)
    {
      shell.exit_status = execute_pipeline(cmd_list, &shell); // Person B
      free_cmd_list(cmd_list); // Person A
    }

    free(input);
  }

  cleanup_shell(&shell);
  return (shell.exit_status);
}
```

🧪 Testing Strategy

📋 Test Categories

1. **Unit Tests** (Each person tests their modules)
2. **Integration Tests** (Test interface between modules)
3. **System Tests** (Test complete shell functionality)
4. **Edge Case Tests** (Empty input, long commands, etc.)

📝 Test Script Examples

```bash
# Basic functionality
echo "hello world"
pwd
ls -la
echo $USER

# Pipes
ls | grep .c
cat file.txt | head -5 | tail -2

# Redirections
echo "test" > output.txt
cat < input.txt > output.txt
cat << EOF

# Complex combinations
echo "start" | cat -n > numbered.txt && echo "done"
```

💡 Pro Tips for Success

🎯 Development Best Practices

1. **Commit early, commit often** - Small, focused commits
2. **Write tests first** - Test-driven development
3. **Code reviews** - Review each other's pull requests
4. **Documentation** - Comment complex logic
5. **Memory management** - Always test with valgrind

🔄 Collaboration Tips

1. **Daily standups** - 15min sync meetings
2. **Pair programming** - For complex integration points
3. **Shared debugging** - Help each other with difficult bugs
4. **Code style** - Agree on formatting standards

🚨 Common Pitfalls to Avoid

1. **Not handling edge cases** - Empty input, invalid syntax
2. **Memory leaks** - Especially with string operations
3. **File descriptor leaks** - Always close opened fds
4. **Signal handling** - Don't ignore Ctrl+C behavior
5. **Process zombies** - Always wait for child processes

📊 Progress Tracking

- Use GitHub issues/projects for task tracking
- Weekly progress reviews
- Regular integration testing
- Performance benchmarking

🎯 Success Metrics

✅ Must Have Features

- [x] Execute simple commands
- [x] Handle pipes (|)
- [x] Handle redirections (<, >, >>)
- [x] Handle heredoc (<<)
- [x] Implement all required builtins
- [x] Handle signals properly
- [x] No memory leaks

🌟 Bonus Features (If time permits)

- [ ] Command history with arrow keys
- [ ] Tab completion
- [ ] Advanced variable expansion
- [ ] Job control (background processes)
- [ ] Wildcard expansion (*)

This plan ensures both team members contribute meaningfully while building a robust, maintainable minishell that meets all 42 requirements!